

Interview Questions and Short Answers

Interview Questions and Short Answers

Project Summary

In my recent project, I built a Playwright-based automation framework in Python for SauceDemo. The framework follows the Page Object Model (POM), which keeps UI elements and test logic separate and makes the tests easier to maintain. I covered core user flows such as login, product selection, cart actions, checkout, and negative test cases.

Interview-Style Answers

1. Briefly describe your project.

In my recent project, I built a Playwright automation framework using Python and pytest. It uses the Page Object Model to keep UI interactions separate from test logic, and it validates key user journeys such as login, add-to-cart, and checkout on the SauceDemo website.

Project-based example:

- The test flow starts from tests/test_login.py and tests/test_checkout.py.
- The page behavior is handled in pages/login_page.py, pages/inventory_page.py, and pages/checkout_page.py.
- The shared setup is managed in tests/conftest.py and config/settings.py.

2. What is a bottleneck or code smell in your project?

One main code smell is that some test data and selectors are still hardcoded, which makes the framework less flexible and harder to scale as the test suite grows.

Project-based example:

- In tests/test_checkout.py, values such as PRODUCT_NAME, CHECKOUT_FIRST_NAME, CHECKOUT_LAST_NAME, and CHECKOUT_POSTAL_CODE are hardcoded directly in the test.
- In pages/login_page.py, the login credentials are embedded in methods like sign_in_standard() and sign_in_bad_password().

Example I would change:

```
```python
Before
PRODUCT_NAME = "Sauce Labs Backpack"

After
product_name = TestData.product_name
...```
```

3. What would you refactor first?

If I were to refactor it today, I would focus on reducing hardcoded values, improving wait strategies, removing duplicate helper logic, and adding better reporting to make the framework more stable and maintainable.

Project-based example:

- I would refactor the repeated page actions in pages/base\_page.py into more reusable helper methods.
- I would also improve the test data handling in tests/test\_checkout.py and tests/test\_login.py.

#### 4. How would you reduce hardcoded values?

I would move remaining test data, environment URLs, and credentials into external configuration files or environment variables so the framework is easier to reuse across different environments.

Project-based example:

- In config/settings.py, the base URL and credentials are already centralized, which is good.
- I would take one step further by moving test-specific values from tests/test\_checkout.py into a separate data file such as data/test\_data.py.

Example I would change:

```
```python
# In tests/test_checkout.py
CHECKOUT_FIRST_NAME = "Tuhin"

# Better approach
from data.test_data import TestData
first_name = TestData.checkout_first_name
```
```

#### 5. How would you handle flakiness?

I would replace static sleep-based waits with dynamic explicit waits so the tests are more reliable and less dependent on timing issues.

Project-based example:

- The current framework uses Playwright expectations in pages/base\_page.py, which is a strong start.
- I would make the waits more explicit around navigation and element visibility in methods such as is\_open() and add\_and\_open\_cart().

Example I would change:

```
```python
# In pages/base_page.py
def expect_visible(self, selector):
    expect(self.page.locator(selector)).to_be_visible()

# I would add a stronger reusable wait helper
def wait_for_page_ready(self):
    self.page.wait_for_load_state("networkidle")
```
```

...

## 6. How would you remove duplicate code?

I noticed repeated helper logic for common actions such as login and navigation, so I would centralize those into reusable utility methods or classes to keep the code cleaner.

Project-based example:

- In pages/login\_page.py, methods such as sign\_in\_standard(), sign\_in\_locked(), and sign\_in\_bad\_password() all use the same sign\_in() method.
- I would further simplify this by using a single method that accepts a user type and password, reducing repetition.

Example I would change:

```
```python
# Before
def sign_in_standard(self):
self.sign_in(Settings.STANDARD_USER, Settings.PASSWORD)

# After
def sign_in(self, username, password):
self.fill(self.USERNAME_INPUT, username)
self.fill(self.PASSWORD_INPUT, password)
self.click(self.LOGIN_BUTTON)
```
```

## 7. How would you improve reporting?

I would integrate a reporting tool such as Allure to provide better failure details, screenshots, and test history, which would make debugging much easier for the team.

Project-based example:

- Right now, the project already saves screenshots and reports through Playwright, but I would enhance it with richer reporting in pytest.
- I would add Allure steps for actions like login, add to cart, and checkout so failures are easier to understand.

Example I would change:

```
```python
import allure

@allure.step("Login with standard user")
def test_login_with_standard_user(navigate_base_url):
login_page = LoginPage(navigate_base_url)
login_page.sign_in_standard()
```
```

## 8. How would you describe your automation approach?

My approach is to keep tests simple, readable, and maintainable by separating page interactions from test assertions and using reusable page objects for each workflow.

Project-based example:

- In this project, each page has its own class, such as LoginPage, InventoryPage, and CartPage.
- The test cases in tests/test\_login.py and tests/test\_checkout.py focus on behavior, while the page classes manage the UI actions.